L'EEPROM du PIC16F84A

Polo Antoine

1. INTRODUCTION

Le PIC16F84A possède une EEPROM de 64 octets dont les adresses vont de 0x00 à 0x3F. Les données y sont persistantes, elles sont conservées même lorsque le microcontrôleur n'est plus alimenté, contrairement à la RAM. Elles ne peuvent cependant pas être lues directement, il faut passer par des registres dédiés.

Dans ce document, nous allons voir comment lire et écrire dans l'EEPROM, puis l'utiliser pour sauvegarder l'état d'une LED et le retrouver au redémarrage.

2. REGISTRES IMPORTANTS

2.1. Les registres de l'EEPROM

Pour fonctionner, l'EEPROM utilise 4 registres :

- `EEADR` : c'est le registre d'adresse, il contient l'adresse de l'octet à écrire ou à lire, de 0 à 63.
- `EEDATA` : c'est le registre de données, il contient l'octet à écrire ou l'octet lu.
- `EECON1` : c'est le registre de commande.
- `EECON2` : c'est un registre fantôme qui sert de sécurité. Il n'existe pas physiquement et sert uniquement à valider une écriture.

2.2. Détail du registre EECON1

Table 1. Bits du registre EECON1.

b7	b6	b5	b4	b3	b2	b1	b0
x	x	x	EEIF	WRERR	WREN	WR	RD

- `EEIF` : c'est le flag d'interruption, il passe à 1 quand l'écriture est terminée. Il doit être remis à 0 par le programme, et déclenche une interruption si `EEIE` et `GIE` de `INTCON` sont à 1.
- `WRERR` : il passe à 1 si l'écriture a été interrompue prématurément (reset ou watchdog).
- `WREN` : s'il est à 0, toutes les écritures sont bloquées ; s'il est à 1, le système est prêt pour une écriture imminente.
- `WR` : le passer à 1 démarre l'écriture. Il repasse automatiquement à 0 quand l'écriture est terminée.
- `RD` : le passer à 1 lance la lecture. Il se remet à 0 automatiquement.

Avant de pouvoir écrire dans l'EEPROM, il faut charger les valeurs 0x55 et 0xAA dans le registre `EECON2`.

2.3. Séquences de lecture et d'écriture

Une lecture se fait en deux étapes :

- écrire l'adresse dans `EEADR` ;
- mettre `RD` à 1, la donnée est alors disponible immédiatement dans `EEDATA`.

Une écriture est plus longue, car elle doit être protégée contre les écritures accidentelles :

- écrire l'adresse dans `EEADR` et la donnée dans `EEDATA` ;
- mettre `WREN` à 1 pour autoriser l'écriture ;
- charger 0x55 puis 0xAA dans `EECON2`, et mettre `WR` à 1 juste après ;
- attendre que `WR` repasse à 0, l'écriture dans l'EEPROM étant lente ;
- remettre `WREN` à 0 pour bloquer à nouveau les écritures.

Note

La séquence 0x55, 0xAA, `WR` doit se dérouler sans interruption. C'est pourquoi les interruptions sont coupées avec `GIE = 0` pendant ces instructions, puis réactivées.

3. EXEMPLE DE CODE

3.1. Montage

Le montage reprend le circuit de base de l'introduction au PIC16F84A (quartz de 8 MHz, condensateurs de 22 pF et résistance de 10 kΩ sur MCLR), auquel on ajoute :

- une LED, avec sa résistance de limitation, sur la broche RB0 ;
- un bouton poussoir entre la broche RB1 et la masse, sans résistance externe puisque le pull-up interne du PORTB est activé.

3.2. Code

Le code permet de changer l'état d'une LED et de le sauvegarder dans l'EEPROM pour remettre la LED dans le même état au redémarrage.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config CP = OFF

#include <xc.h>

#define _XTAL_FREQ 8000000

#define LED_STATUS PORTBbits.RB0
#define BUTTON_TOGGLE PORTBbits.RB1

#define ADDR_LED_STATE 0x00

uint8_t eeprom_read(uint8_t addr) {
    EEADR = addr;
    EECON1bits.RD = 1;
    return EEDATA;
}

void eeprom_write(uint8_t addr, uint8_t data) {
    EEADR = addr;
    EEDATA = data;
    EECON1bits.WREN = 1;

    INTCONbits.GIE = 0;
    EECON2 = 0x55;
    EECON2 = 0xAA;
    EECON1bits.WR = 1;
    INTCONbits.GIE = 1;

    while (EECON1bits.WR);

    EECON1bits.WREN = 0;
}

void main(void) {
    TRISB = 0b00000010;
    OPTION_REGbits.nRBPU = 0;

    uint8_t state_led = 0;

    uint8_t saved = eeprom_read(ADDR_LED_STATE);
    state_led = (saved == 1) ? 1 : 0;
    LED_STATUS = state_led;

    uint8_t last_button_state = 1;

    while (1) {
```

```
        uint8_t cur_button_state = BUTTON_TOGGLE;

        if (cur_button_state == 0 && last_button_state == 1)
    ↪ {
            state_led = !state_led;
            LED_STATUS = state_led;
            eeprom_write(ADDR_LED_STATE, state_led);
        }

        last_button_state == cur_button_state;
    }
}
```

Code 1. Sauvegarde de l'état d'une LED dans l'EEPROM.

3.3. Fonctionnement

- **Définitions** : `LED_STATUS` et `BUTTON_TOGGLE` donnent un nom aux broches RB0 et RB1, et `ADDR_LED_STATE` est l'adresse de l'EEPROM où l'état de la LED est stocké.
- **Fonctions de lecture et d'écriture** : `eeprom_read` et `eeprom_write` suivent exactement les séquences décrites plus haut.
- **Configuration** : `TRISB = 0b00000010` configure RB1 en entrée (le bouton) et toutes les autres broches du PORTB en sorties, dont RB0 (la LED). `nRBPU = 0` active les pull-up du PORTB : RB1 vaut 1 au repos et 0 lorsque le bouton est appuyé.
- **Démarrage** : l'état sauvegardé est lu à l'adresse `ADDR_LED_STATE`, puis appliqué à la LED. Le test `(saved == 1) ? 1 : 0` évite de se tromper avec une EEPROM vierge, qui contient 0xFF : la LED reste alors éteinte.
- **Boucle principale** : on compare l'état actuel du bouton à l'état précédent. Un appui est détecté quand le bouton passe de 1 à 0. La LED est alors inversée et le nouvel état est écrit dans l'EEPROM.

Note

Une EEPROM supporte un nombre limité d'écritures. Ce programme écrit à chaque appui sur le bouton, ce qui est suffisant pour un exemple, mais il vaut mieux limiter les écritures dans un vrai projet.

4. NOTES COMPLÉMENTAIRES

Les fonctions `eeprom_read` et `eeprom_write` ont été définies à titre d'exemple. Elles sont disponibles de façon native dans le compilateur XC8.

5. CONCLUSION

L'EEPROM permet de conserver des données, comme un réglage ou un état, même lorsque le microcontrôleur est éteint. Sa lecture est simple, alors que son écriture demande une séquence précise (`WREN`, 0x55 et 0xAA dans `EECON2`, puis `WR`) qui la protège contre les écritures accidentelles.